A Dual-Encoding-based Genetic Algorithm for Multi-Objective Multi-UAV Scheduling in Firefighting Scenarios

*

***Abstract*—Unmanned aerial vehicles (UAVs) are increasingly used for firefighting due to security. Multiple UAVs depart from a site and cooperate to execute tasks with time-varying demands in different locations. To better execute firefighting tasks, the reasonable scheduling of UAVs is crucial. Although multiple methods have been developed to solve the multi-UAV scheduling problem, they still face challenges in scenarios with large-scale tasks. This paper models the problem as a bi-objective optimization problem, aiming to minimize two important objectives: the makespan and the total travel distance of UAVs. A dual-encoding-based genetic algorithm (DEGA) is developed to address the problem. In DEGA, a dual encoding scheme is adopted for solution representation, in which the execution order of all tasks is represented as a sequence and the task assignment for UAVs is represented as a binary matrix. Correspondingly, solutions can be constructed in two steps: task sequence generation first and then task assignment. Crossover and mutation operations are specifically designed to explore the search space for enhancing solution diversity. In addition, a local search is employed to improve solution quality. Experimental results on instances with various scales demonstrate that DEGA significantly outperforms state-of-the-art algorithms in terms of solution optimality and diversity.**

***Index Terms*—Genetic algorithm, multi-objective optimization, multi-UAV scheduling, firefighting, multi-robot systems.**

I. Introduction

Unmanned aerial vehicles (UAVs) are widely used in real-world applications, such as monitoring [1], communication [2], and goods delivery [3]. Particularly in firefighting, multiple UAVs are equipped with fire-extinguishing devices and form a multi-agent system to eliminate bushfires. Each UAV is assigned with multiple tasks and plans paths to visit the task positions to extinguish fires as quickly as possible [4]. Compared with a single UAV, multi-UAV systems have higher stability and efficiency [5], [6]. In such systems, the scheduling of UAVs for task execution is an important issue, involving task assignment and path planning of each UAV.

To reduce damage and costs in firefighting, the makespan and traveling distances of UAVs are two important factors for decision-making, which, in certain scenarios, can conflict with each other. Thus, the scheduling problem can be formulated as a multi-objective optimization problem [7]. As the fire spreads, the demand at each fire location changes over time. This increases the difficulty of the scheduling of UAVs. In the literature, multiple evolutionary computation methods are developed to solve this problem due to the good performance on multi-objective optimization, such as genetic algorithms (GA) [8], particle swarm optimization (PSO) [9], and ant colony optimization (ACO) [10].

A hybrid decomposition-based multi-objective evolutionary algorithm adopts the ϵ-constraint method to decompose the problem into subproblems [11]. In addition, a multi-strategy genetic algorithm (MSGA) employs multiple genetic operations and repair mechanisms to address complex scheduling constraints [12]. However, these operators are inefficient in a large search space. A pseudo gradient-adjusted PSO is proposed for adaptive latent factor analysis [13]. Local search is further adopted to improve search efficiency [14]. Adaptive coordination ant colony optimization (ACACO) represents solutions as multiple independent task sequences of UAVs and adopts multiple pheromone matrices to record the task sequences for solution construction [15]. However, this encoding scheme easily causes conflicts in task execution since two tasks may be assigned to two distant UAVs in an opposite execution order locally. Genetic programming is also explored for evolving coordination strategies in dynamic environments [16], [17].

Though existing methods perform well on small-scale problems, they face challenges on large-scale problems in terms of solution diversity and convergence. Thus, this paper proposes a dual-encoding-based genetic algorithm (DEGA) to solve the multi-objective multi-UAV scheduling problem. In DEGA, a dual-encoding scheme is adopted for solution representation, in which the execution order of all tasks is represented as a sequence and the assignment of tasks to UAVs is represented as a binary matrix. Solutions are generated by first planning the task sequence from a global perspective and then assigning the tasks to UAVs. Crossover and mutation operators with repair mechanisms are specially designed for the dual-encoding-based solutions to enhance solution diversity. In addition, a local search is developed to further improve solution convergence. Experiments are performed on 28 instances with different scales and the results show that the proposed method performs significantly better than the state-of-the-art algorithms in terms of solution optimality and diversity.

Identify applicable funding agency here. If none, delete this.

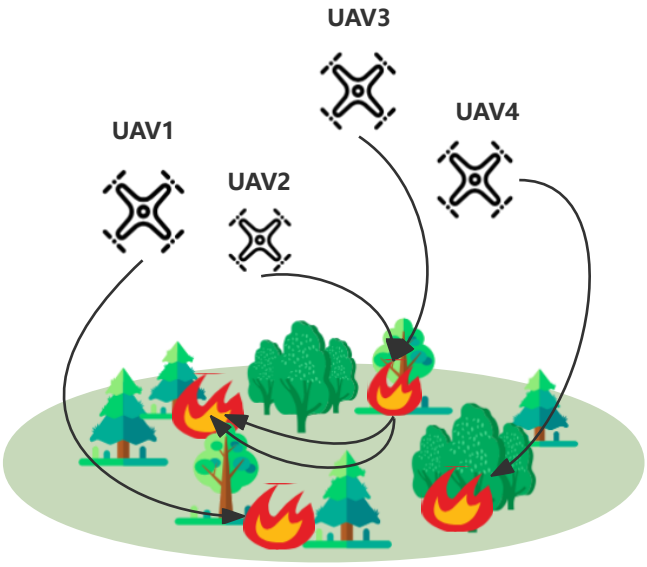


Fig. 1: Example of a bushfire elimination problem.

The remainder of the paper is organized as follows. Section II gives the problem formulas. Section III presents the proposed DEGA. Section IV reports the experimental results and section V summaries conclusions.

II. Background

In a bushfire elimination problem, multiple tasks (e.g., fire points) are distributed at different locations. Each task has a time-varying demand. Multiple UAVs depart from a site and travel to visit multiple task locations for eliminating fires as Fig. 1. Denote the site location as t_0, the task set as $T = \{t_j | 1 \leq j \leq N\}$ and the UAV set as $U = \{u_i | 1 \leq i \leq M\}$, where N is the number of tasks and M is the number of UAVs. Since the fire gets bigger with time, the outfire demand of a task t_j changes as

$$q_j(t) = q_j(0) + \gamma_j t \tag{1}$$

where t is the time, $q_j(0)$ is the initial demand, and γ_j is the growth rate. Each UAV u_i has a capability to eliminate fires per unit time, which is denotes as μ_i.

To finish all tasks, a scheduling solution is found to plan the task execution paths of all UAVs. A solution is denoted as $X = \{x^1, ..., x^M\}$, where $x^k = [x^k_{ij}]$ is a $N \times N$ matrix to record the task sequence of u_k. Particularly, $x^k_{ij} \in \{0, 1\}$ represents that whether u_k travels from the task t_i to t_j. If $x_{ij} = 1$, u_k travels from the task t_i to t_j; otherwise, the opposite. The capacity value of assigned UAVs along the execution time must fulfill the task demand. The complete time e_j of task t_j meets:

$$\begin{aligned} q_j(0) + \gamma_j e_j &= \sum_{i=0}^{N} \sum_{k=1}^{M} \mu_k (e_j - a_{jk}) x^k_{ij} \\ a_{jk} &= \sum_{i=0}^{N} (e_i + t_{ij}) x^k_{ij} \end{aligned} \tag{2}$$

where a_{jk} is the arriving time of UAV u_k at the task t_j, and t_{ij} is the traveling time from t_i to t_j. The traveling distance of u_k is calculated as:

$$d^k = \sum_{i=0}^{N} d_{ij} x^k_{ij} \tag{3}$$

where d_{ij} is the distance between t_i and t_j. There are three constraints, i.e., task assignment, task execution order, and task execution times. Particularly, each task must be executed by at least one UAV as

$$\sum_{k=1}^{M} \sum_{i=0}^{N} x^k_{ij} \geq 1, \quad \forall 1 \leq j \leq N \tag{4}$$

The execution order of tasks must be consistent for the assigned UAVs as

$$\sum_{i=0}^{N} x^k_{ij} = \sum_{i=0}^{N} x^k_{ji}, \quad \forall 1 \leq j \leq N, 1 \leq k \leq M \tag{5}$$

Each UAV is allowed to execute a specific task at most once:

$$\sum_{i=0}^{N} x^k_{ij} \leq 1, \quad \forall 1 \leq j \leq N, 1 \leq k \leq M \tag{6}$$

Two optimization objectives are considered: minimizing the makespan and the traveling distances of all UAVs. Particularly, the makespan is the maximum complete time among all tasks as

$$f_1(X) = \max_{1 \leq j \leq N} e_j \tag{7}$$

The total traveling time of all UAVs is calculated as

$$f_2(X) = \sum_{i=1}^{M} d^k \tag{8}$$

Thus, the scheduling problem can be formulated as bi-objective optimization problem as:

$$\min \{f_1(X), f_2(X)\} \tag{9}$$

Optimization algorithms are required to find the Pareto front of the problem.

To better illustrate this problem, let us consider a simple example. Suppose there are two fire points in a forest that need to be addressed by two UAVs. The UAVs are represented as u_1 and u_2, and the tasks are represented as t_1 and t_2. The UAVs move through the forest at a velocity $v = 20\ km/h$ with a fire elimination capability $\mu_1 = \mu_2 = 40\ units/h$. t_1 and t_2 have initial demand of $q_1(0) = 300\ units$ and $q_2(0) = 100\ units$, with growth rates of $\gamma_1 = 30\ units/h$ and $\gamma_2 = 10\ units/h$, respectively. Both tasks are located 10 km from the UAVs' starting position ($D_1 = D_2 = 10\ km$), and the distance between the tasks is also 10 km. Task t_1 has both higher initial demand and growth rate than task t_2, thus requiring more urgent attention.

Three potential strategies are proposed: Strategy I dispatches one UAV to each task simultaneously to address both tasks at the same time. Strategy II deploys both UAVs to task t_1 first

to extinguish it, and then both proceed together to task t_2. Strategy III starts by sending one UAV to each task, but after completing task t_2, the UAV assigned there returns to assist the other UAV at task t_1.

Let us derive the elimination time and travel distances for these strategies. For Strategy I, with u_1 traveling to task t_1 and u_2 traveling to task t_2, the solution is:

$$X = \{x^1, x^2\} = \left\{ \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \right\} \tag{10}$$

where $x^1_{01} = 1$ and $x^1_{10} = 1$ indicate that u_1 travels from the starting point to fire point 1 and return to the starting point, respectively. Similarly, $x^2_{02} = 1$ and $x^2_{20} = 1$ indicate that u_2 travels from the starting point to fire point 2 and return to the starting point, respectively. The arrival time and travel time of the UAVs are:

$$a_{11} = a_{22} = \frac{D_1}{v} = \frac{D_2}{v} = 0.5 \ h \tag{11}$$

where a_{11} is the arrival time of u_1 at fire point 1, a_{22} is the arrival time of u_2 at fire point 2. The two times are equal and are 0.5 h. In addition, the completion times e_1 and e_2 of tasks t_1 and t_2 meet:

$$q_1(0) + \gamma_1 e_1 = \mu_1(e_1 - a_{11}) \tag{12}$$

$$q_2(0) + \gamma_2 e_2 = \mu_2(e_2 - a_{22}) \tag{13}$$

We have calculated all the values except e_1 and e_2, so it is easy to calculate e_1 and e_2 as follows:

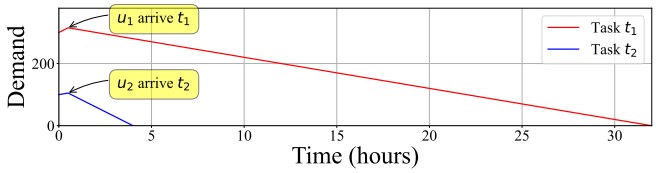
$$e_1 = \frac{q_1(0) + \mu_1 a_{11}}{\mu_1 - \gamma_1} = 32 \ h \tag{14}$$

$$e_2 = \frac{q_2(0) + \mu_2 a_{22}}{\mu_2 - \gamma_2} = 4 \ h \tag{15}$$

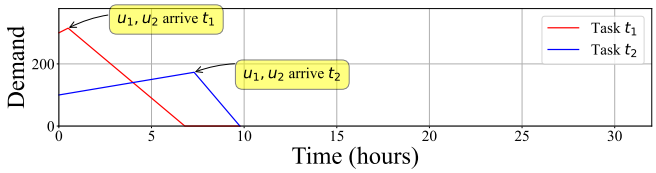
The makespan is the maximum of the completion times, i.e., $f_1(X) = \max(e_1, e_2) = 32 \ h$. The total travel distance is $f_2(X) = d^1 + d^2 = 2 * D_1 + 2 * D_2 = 40 \ km$.

Similarly, we can calculate the elimination time and travel distance for Strategies II and III. For Strategy II, the fitness values are $\{f_1(X), f_2(X)\} = \{9.77 \ h, 60 \ km\}$. For Strategy III, the fitness values are $\{f_1(X), f_2(X)\} = \{10 \ h, 50 \ km\}$. These three strategies are illustrated in Fig. 2.

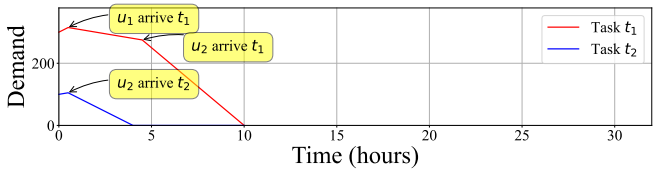
In Fig. 2, the x-axis represents time, and the y-axis represents the demand of the tasks. The red lines represent the task demand changes of task t_1, while blue lines show those of task t_2. The turning points in these lines indicate UAV arrival events. Strategy I minimizes travel distance but requires the longest elimination time; Strategy II achieves the shortest elimination time but involves the longest travel distance; Strategy III presents intermediate values for both metrics. Notably, none of these solutions completely dominates the others. This demonstrates that elimination time and travel distance can be conflicting objectives, justifying the formulation of this problem as a multi-objective optimization problem with makespan and total travel distance as the objective functions.



(a) Strategy I: $\{f_1(X), f_2(X)\} = \{32, 40\}$



(b) Strategy II: $\{f_1(X), f_2(X)\} = \{9.77, 60\}$



(c) Strategy III: $\{f_1(X), f_2(X)\} = \{10, 50\}$

Fig. 2: Illustration of three strategies for a simple case.

III. Method

The proposed DEGA employs a dual-encoding scheme for solution representation. Solutions are generated in two steps: task sequence construction first and then task assignment. Crossover and mutation operators are used to enhance solution diversity. A local search is performed to improve convergence. The details of each component are introduced in the following. The overall flowchart of DEGA is illustrated in Fig. 3.

A. Solution Representation

Since the constraints, Solutions are represented using a dual-encoding scheme, which encodes the task assignment as a binary matrix and the visiting path of each UAV as a task sequence.

1) Task Sequence Encoding: The execution order of all tasks is represented as a task permutation $\mathbf{R} = [r_1, r_2, \ldots, r_N]$, where N is the total number of tasks and each r_i is ith task for execution. For the tasks assigned to a UAV, these tasks will be executed as their order in the task permutation.

2) Task Assignment Encoding: Task assignment for UAVs is represented as a binary matrix $\mathbf{X} = [x_{ij}]$ with a size of $M \times N$. Each element $x_{ij} \in \{0, 1\}$ indicates whether task t_j is assigned to UAV u_i:

$$x_{ij} = \begin{cases} 1, & \text{if task } t_j \text{ is assigned to UAV } u_i, \\ 0, & \text{otherwise.} \end{cases} \tag{16}$$

Each UAV executes the tasks assigned to it (as defined in $\mathbf{X}$) following the order in the task sequence $\mathbf{R}$. In this way, all UAVs execute the assigned tasks in the same order for meeting the constraints, potentially leading to more efficient planning.

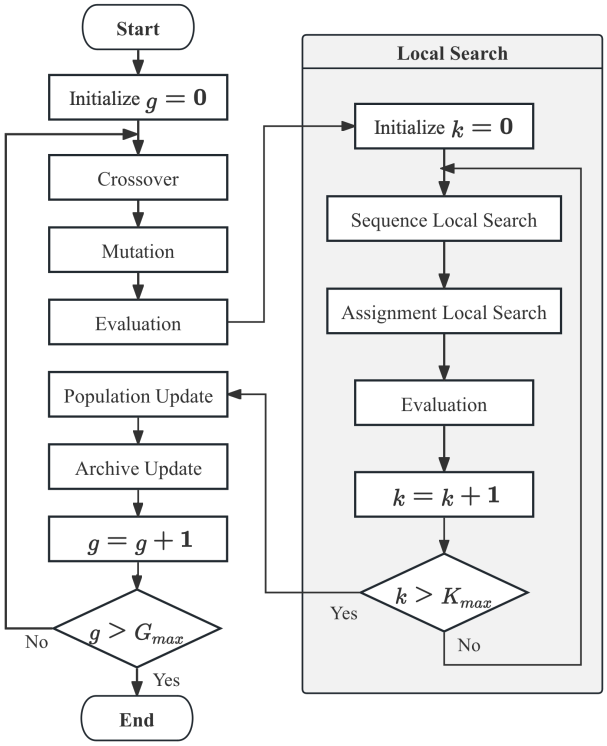


Fig. 3: The flowchart of the proposed DEGA.

Algorithm 1 Crossover

Input: Parent individual ($\mathbf{R}_1$, $\mathbf{X}_1$), ($\mathbf{R}_2$, $\mathbf{X}_2$)
Output: Offspring individual ($\mathbf{R}_{\text{offspring}}$, $\mathbf{X}_{\text{offspring}}$)

// Global Task Sequence Crossover
Randomly select crossover points c_1 and c_2 such that $1 \leq c_1 < c_2 \leq N$
Initialize offspring sequence $\mathbf{R}_{\text{offspring}}$ with placeholders
Copy subsequence: $\mathbf{R}_{\text{offspring}}[c_1 : c_2] \leftarrow \mathbf{R}_1[c_1 : c_2]$
Initialize index $j \leftarrow c_2 + 1$
for i from $c_2 + 1$ to N, then from 1 to c_2 (modulo N) **do**
  if $\mathbf{R}_2[i]$ not in $\mathbf{R}_{\text{offspring}}$ **then**
    $\mathbf{R}_{\text{offspring}}[j] \leftarrow \mathbf{R}_2[i]$
    $j \leftarrow j \mod N + 1$
  end if
end for
// Task Assignment Crossover
Randomly select an operator *op* from {"+", "-", "×"}
if *op* is "+" **then**
  $\mathbf{X}_{\text{offspring}} \leftarrow \min(\mathbf{X}_1 + \mathbf{X}_2, 1)$
else if *op* is "-" **then**
  $\mathbf{X}_{\text{offspring}} \leftarrow |\mathbf{X}_1 - \mathbf{X}_2|$
else if *op* is "×" **then**
  $\mathbf{X}_{\text{offspring}} \leftarrow \mathbf{X}_1 \times \mathbf{X}_2$
end if
// Repair Mechanism
Repair the mutated $\mathbf{X}_{\text{offspring}}$ to ensure all constraints are satisfied
return ($\mathbf{R}_{\text{offspring}}$, $\mathbf{X}_{\text{offspring}}$)

B. Genetic Operators

DEGA employs specialized genetic operators based on the dual-encoding scheme. These operators are designed to enhance diversity.

1) Crossover: For the global task sequence $\mathbf{R}$, a modified order crossover is applied. Given two parent sequences $\mathbf{R}_1$ and $\mathbf{R}_2$, two crossover points are randomly selected first, denoted as c_1 and c_2 ($1 \leq c_1 < c_2 \leq N$). Denote a new task sequence as $\mathbf{R}_{\text{offspring}}$. The subsequence between c_1 and c_2 in $\mathbf{R}_1$ is selected to add in $\mathbf{R}_{\text{offspring}}$. The unvisited tasks in the offspring $\mathbf{R}_{\text{offspring}}$ are inserted in the same order as that in $\mathbf{R}_2$.

For the task assignment matrix $\mathbf{X}$, DEGA uses arithmetic operations to combine parent matrices. Given two parent matrices $\mathbf{X}_1$ and $\mathbf{X}_2$, the offspring matrix $\mathbf{X}_{\text{offspring}}$ is generated using operations such as addition, subtraction, or multiplication. Infeasible offsprings are repaired to meet constraints. **Algorithm 1** outlines the procedure of crossover.

2) Mutation: To introduce diversity and explore new regions of the search space, DEGA applies mutation on every offspring with a probability of p_m. The mutation involves two primary operations: modifying the task sequence and altering the task assignments. For the task sequence of an offspring, either a swap or an insertion mutation is performed. That is, two random tasks are exchanged or a random task is moved to a different position in the sequence.

Concurrently, for the task assignment matrix of the offspring, a random integer $a \in [1, M]$ is generated first as the flipping times. In each flipping operator, a random integer $b \in [1, N]$ is generated first, and then a UAV and b tasks are randomly selected to flip their assignment status, i.e., $x_{ij} \leftarrow 1 - x_{ij}$ for the selected UAV u_i and every selected task t_j. The mutation procedure is shown in **Algorithm 2**. The mutation helps to enhance solution diversity.

3) Repair Mechanism: After the crossover and mutation, new offsprings are repaired to ensure that all constraints are satisfied. For each task t_j, if it is not yet assigned to any UAV, UAVs that are not assigned to t_j will be randomly selected and assigned to t_j until their combined capability exceeds the growth rate γ_j of the task demand. That is, the combined capability of UAVs assigned to task t_j must satisfy the constraint:

$$\sum_{u_i \in U_j} \mu_i \geq \gamma_j \tag{17}$$

where U_j is the set of UAVs assigned to task t_j. This ensures that the outfire demand at each task location is met over time.

C. Local Search

To refine solutions and improve convergence, DEGA incorporates a local search to adjust the global task sequence and the task assignment matrix.

1) Global Task Sequence: The local search explores neighboring solutions by making small adjustments to the task

Algorithm 2 Mutation

Input: Individual ($\mathbf{R}$, $\mathbf{X}$)
Output: Mutated individual ($\mathbf{R}'$, $\mathbf{X}'$)

// Task Sequence Mutation
Randomly decide to perform Swap or Insertion Mutation
if Swap Mutation **then**
  Randomly select two positions i and j in $\mathbf{R}$
  Swap tasks at positions i and j
else if Insertion Mutation **then**
  Randomly select a task at position i and insert it at position j
end if
// Task Assignment Mutation
Randomly generate integers $a \in [1, M]$
for each k from 1 to a **do**
  Randomly generate integers $b \in [1, N]$
  for each l from 1 to b **do**
    Randomly select a UAV u_i and a tasks t_j
    Flip the assignment: $x_{ij} \leftarrow 1 - x_{ij}$
  end for
end for
// Repair Mechanism
Repair the mutated $\mathbf{X}'$ to ensure all constraints are satisfied
return Mutated and repaired individual ($\mathbf{R}'$, $\mathbf{X}'$)

Algorithm 3 Dual-Encoding-based Genetic Algorithm

Input: Problem data (UAVs, tasks, distances), parameters ($G_{\max}$, population size N_p, crossover rate p_c, mutation rate p_m)
Output: An archive $\mathcal{A}$

Initialize population $\mathcal{P}$ with N_p individuals using the dual-encoding scheme
Evaluate fitness of each individual in $\mathcal{P}$
Initialize Pareto archive $\mathcal{A}$ with non-dominated individuals from $\mathcal{P}$
for $g = 1$ to $G_{\max}$ **do**
  Select parent individuals from $\mathcal{P}$
  Apply crossover operators with probability p_c to generate offsprings
  Apply mutation operators on the offsprings with probability p_m
  Repair the infeasible offsprings
  Apply local search to offsprings
  Evaluate fitness of offsprings
  Update $\mathcal{A}$ with non-dominated solutions among the combination of offsprings and the population
  Update the population $\mathcal{P}$ by selecting individuals from the population and offsprings based on nondominated sorting and crowding distance sorting
end for
return Pareto archive $\mathcal{A}$

sequence $\mathbf{R}$. Two tasks in the sequence are randomly selected for swapping. A subsequence is randomly selected to reverse the task order. This helps to find a better task sequence for reducing the total travel distance of all UAVs.

2) Task Assignment: For the assignment matrix $\mathbf{X}$, a task assigned to a UAV is reassigned to another one, considering the capacities and capabilities of the UAVs. In addition, two tasks assigned to two distant UAVs are swapped. This fine-tuning can balance the workload among UAVs and potentially reduce the makespan.

At the end of each iteration, all new solutions are combined with the population and the non-dominated ones are reserved based on the crowding distance sorting to maintain diversity.

D. The Complete Algorithm

The complete algorithm is presented in **Algorithm 3**. At the beginning, an initial population $\mathcal{P}$ of a size N_p is randomly generated. Infeasible individuals are repaired via the repair mechanism. All individuals are evaluated and the nondominated solutions are stored in an extra Pareto archive $\mathcal{A}$. In each generation, individuals are randomly selected from the population as parents. Crossover is performed on parents with a probability of p_c and mutation is applied with a probability of p_m to generate offsprings. After that, the repair mechanism is performed on the offsprings to meet constraints. Later, local search is applied to the offsprings. At the end of each generation, all offsprings are combined with the population and the archived solutions for nondominated sorting. The nondominated ones are used to update the archive $\mathcal{A}$. The population is updated by selecting individuals from the combined set of parents and offspring according to the non-dominated sorting and crowding distance sorting. This procedure repeats until reaching the maximum number of generations $G_{\max}$. Finally, the archive $\mathcal{A}$ is taken as the algorithm output.

IV. Experiment

A. Test Instances

To evaluate the performance of DEGA, the recent instances in [15] are adopted to test. Each instance is characterized by a different combination of UAVs and tasks, providing a diverse set of scenarios to assess the effectiveness of the algorithm under various operational conditions. There are 16 small-scale, 8 medium-scale, and 4 large-scale test instances. The number of UAVs ranges from 3 to 80, and the number of tasks ranges from 4 to 60. Each instance is identified by a specific notation, such as L_20_60_1.51, which represents a scenario with 20 UAVs and 60 tasks. The last number in the notation, 1.51, indicates the ratio of total task demand growth rate to total UAV capacity, providing a measure of the complexity of the scheduling problem relative to available UAV resources. In each instance, UAVs must coordinate to complete a set of distributed tasks efficiently. The complexity of the problem increases as the number of tasks grows, or as the ratio of task demand to UAV capacity rises.

B. Comparison Algorithms

Five representative algorithms for the scheduling problem are selected for comparisons, i.e., iterated local search (ILS)

TABLE I: Comparison of Methods for HV

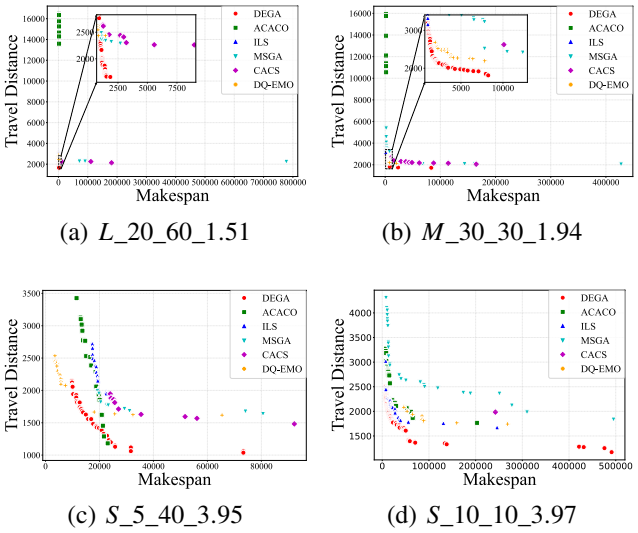
Test Cases	DEGA	ACACO	ILS	MSGA	CACS	DQ-EMO
*L*_20_60_1.51	**3.02E+12**	6.70E+11(+)	2.78E+12(+)	2.95E+12(+)	2.96E+12(+)	2.89E+12(+)
*L*_80_15_0.76	4.68E+12	1.19E+12(+)	4.57E+12(+)	**4.71E+12**(−)	4.71E+12(−)	4.70E+12(−)
*L*_80_20_0.7	2.78E+12	9.11E+11(+)	2.69E+12(+)	**2.91E+12**(−)	2.90E+12(−)	2.85E+12(−)
*L*_80_20_1.12	**1.88E+14**	5.04E+13(+)	1.82E+14(+)	1.87E+14(+)	1.87E+14(+)	1.87E+14(+)
*M*_15_30_2.16	**2.59E+14**	1.50E+14(+)	2.12E+14(+)	2.51E+14(+)	2.50E+14(+)	2.45E+14(+)
*M*_20_20_0.58	3.74E+08	1.16E+08(+)	3.57E+08(+)	3.87E+08(−)	**3.88E+08**(−)	3.76E+08(−)
*M*_20_20_0.967	1.11E+13	3.72E+12(+)	1.02E+13(+)	1.14E+13(−)	**1.14E+13**(−)	1.09E+13(+)
*M*_20_20_0.97	**5.20E+09**	1.42E+09(+)	5.02E+09(+)	5.09E+09(+)	5.09E+09(+)	5.12E+09(+)
*M*_30_30_1.04	**3.10E+11**	8.38E+10(+)	2.83E+11(+)	3.06E+11(+)	3.06E+11(+)	3.02E+11(+)
*M*_30_30_1.94	**2.09E+16**	7.43E+15(+)	1.89E+16(+)	2.06E+16(+)	2.06E+16(+)	2.04E+16(+)
*M*_40_10_0.67	7.03E+10	2.43E+10(+)	6.46E+10(+)	7.41E+10(−)	**7.44E+10**(−)	7.25E+10(−)
*M*_40_15_0.67	1.46E+10	4.01E+09(+)	1.36E+10(+)	1.57E+10(−)	**1.57E+10**(−)	1.52E+10(−)
*S*_10_10_3.79	**3.47E+17**	2.78E+17(+)	2.85E+17(+)	2.98E+17(+)	3.20E+17(+)	2.83E+17(+)
*S*_10_15_1.3	1.17E+10	3.96E+09(+)	9.12E+09(+)	1.15E+10(+)	**1.17E+10**(≈)	1.05E+10(+)
*S*_10_5_1.39	**5.63E+10**	3.47E+10(+)	4.42E+10(+)	5.41E+10(+)	5.51E+10(+)	4.87E+10(+)
*S*_15_10_1.17	**1.21E+11**	2.66E+10(+)	1.06E+11(+)	1.18E+11(+)	1.19E+11(+)	1.18E+11(+)
*S*_15_20_0.67	1.91E+08	6.34E+07(+)	1.63E+08(+)	1.95E+08(−)	**1.97E+08**(−)	1.80E+08(+)
*S*_17_23_1.71	**3.54E+11**	2.04E+11(+)	3.28E+11(+)	3.37E+11(+)	3.35E+11(+)	3.35E+11(+)
*S*_20_10_0.47	1.40E+08	3.71E+07(+)	1.31E+08(+)	**1.41E+08**(−)	1.41E+08(−)	1.39E+08(+)
*S*_20_10_0.94	**6.68E+09**	1.44E+09(+)	6.12E+09(+)	6.66E+09(+)	6.68E+09(≈)	6.63E+09(+)
*S*_30_10_0.65	2.02E+10	5.23E+09(+)	1.95E+10(+)	2.04E+10(−)	**2.05E+10**(−)	2.04E+10(−)
*S*_30_10_1.34	**3.92E+12**	7.52E+11(+)	3.52E+12(+)	3.81E+12(+)	3.85E+12(+)	3.78E+12(+)
*S*_3_10_1.51	9.65E+06	4.78E+06(+)	8.20E+06(+)	9.16E+06(+)	**9.71E+06**(≈)	8.49E+06(+)
*S*_3_15_5.03	**4.09E+12**	3.49E+12(+)	3.26E+12(+)	2.78E+12(+)	3.54E+12(+)	2.13E+12(+)
*S*_5_10_0.93	**1.78E+07**	3.89E+06(+)	1.55E+07(+)	1.66E+07(+)	1.77E+07(≈)	1.60E+07(+)
*S*_5_10_3.67	**3.28E+13**	2.70E+13(+)	2.78E+13(+)	2.69E+13(+)	3.11E+13(+)	2.50E+13(+)
*S*_5_40_3.95	**1.10E+10**	9.38E+09(+)	7.05E+09(+)	9.10E+09(+)	9.54E+09(+)	8.41E+09(+)
*S*_5_4_0.39	3.48E+05	2.07E+04(+)	2.49E+05(+)	**3.56E+05**(≈)	3.31E+05(+)	2.68E+05(+)
Rank Sum Test	–	28 / 0 / 0	28 / 0 / 0	18 / 1 / 9	15 / 4 / 9	22 / 0 / 6

[18], adaptive coordination ant colony optimization (ACACO) [15], multi-strategy genetic algorithm (MSGA) [12], cooperative ant colony system (CACS) [10], and deep Q-learning with evolutionary many-objective optimization (DQ-EMO) [19].

Particularly, ILS is a heuristic algorithm, which improves solutions through iterative perturbations and local search to balance the global exploration and local exploitation. ACACO is designed for the multi-robot fire fighting scheduling problem, enhancing UAV coordination with multiple pheromone matrices and heuristic information. MSGA uses matrix encoding and combines multiple genetic operations with a repair mechanism to address complex constraints. CACS introduces a multi-objective approach using three pheromone matrices to optimize three objectives simultaneously, integrating solution diversity and convergence through a fusion-based local search. Finally, DQ-EMO combines deep Q-learning with evolutionary algorithms to tackle many-objective optimization problems, which use a modified SPEA2 to optimize multiple objectives.

In multi-objective optimization, solutions are evaluated based on multiple criteria, and a solution is considered non-dominated if no other solution is better in all objectives simultaneously. The collection of these non-dominated solutions forms the Pareto front, representing the trade-offs among objectives where improving one objective would worsen another.

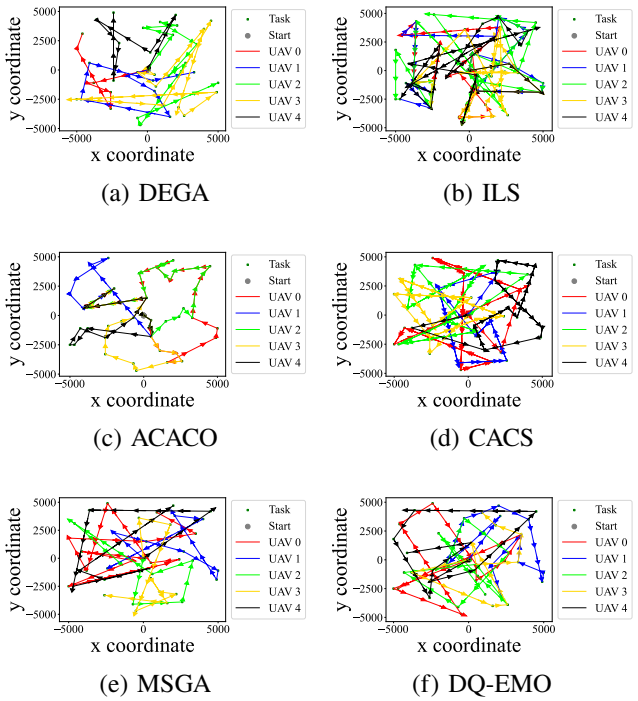
Therefore, the performance of DEGA is evaluated based on the hypervolume of the Pareto front, which evaluates the quality of solutions by measuring the volume of the objective space dominated by the Pareto front relative to a reference point. It reflects both the convergence and diversity of the solutions, making it a widely used metric in multi-objective optimization. To compute the hypervolume, the reference point is set as the 1.1 times of the maximum values of these objectives found by all methods. A larger hypervolume value indicates better algorithm performance. For fairness, the maximum number of evaluations for all algorithms is set as $E = M \times N \times 1.1 \times G_{max}$, where M is the number of UAVs, N is the number of tasks, and $G_{max} = 100$ is the number of iterations. Particularly, $K_{max} = 0.1 \times M \times N$ evaluations are allocated for local search in ACACO and DEGA in each iteration. In DEGA, the population size is set as $M \times N$, the crossover probability is $P_c = 0.8$ and the mutation probability is $P_m = 0.05$.



(a) $L\_20\_60\_1.51$ (b) $M\_30\_30\_1.94$

(c) $S\_5\_40\_3.95$ (d) $S\_10\_10\_3.97$

Fig. 4: Comparison of Pareto fronts in different instances



(a) DEGA (b) ILS

(c) ACACO (d) CACS

(e) MSGA (f) DQ-EMO

Fig. 5: Comparison of robot paths using different algorithms

Each algorithm runs 20 times independently on each test instance. The mean value of the results are reported. Wilcoxon rank-sum test (significance level of 0.05) is performed to test the significance differences between the proposed DEGA and each competing algorithm. The significance results are denoted as "+", "=", and "≈" to represent that DEGA is significantly better than, equal to, and worse than the competing algorithm. The best-performing algorithms are highlighted in bold.

C. Experiment Results

The hypervolume results are presented in Table I. The results demonstrate that DEGA outperforms both ACACO and ILS across all three metrics in 28 instances. DEGA also performs better than MSGA in 18 cases in terms of hypervolume, while being outperformed in only 9 instances. The relatively lower performance of ACACO and ILS can be attributed to their less efficient search strategies, which struggle with complex path planning and task allocation problems. In contrast, MSGA performs better in scenarios where the number of UAVs significantly exceeds the number of tasks, achieving the best hypervolume in 4 instances. CACS and DQ-EMO also deliver competitive results in certain scenarios. Specifically, CACS demonstrates an effective ability to adapt flight routes and maintain consistency in some mid-scale instances, achieving the optimal hypervolume in 8 small- or mid-scale cases. Despite these strengths, DEGA consistently exhibits superior performance in most instances. This can be attributed to its dual-encoding scheme, which enables simultaneous optimization of both task allocation and path planning, thereby enhancing overall optimization effectiveness.In summary, DEGA demonstrates exceptional performance in the bi-objective scheduling problem.

To better visualize the differences among various methods, we analyzed the Pareto fronts of four instances: L_20_60_1.51, M_30_30_1.94, S_5_40_3.95, and S_10_10_3.79, as shown in Fig. 4. The results demonstrate that ACACO performs well in smaller instances (S_5_40_3.95 and S_10_10_3.79), it struggles with larger instances (L_20_60_1.51 and M_30_30_1.94) due to its lower efficiency in handling complex path planning and task allocation problems. ILS shows relatively poor performance across all instances due to its inefficient search strategy. Although MSGA demonstrates slightly better performance than CACS in L_20_60_1.51, M_30_30_1.94, and S_5_40_3.95, showcasing its optimization capability when the number of tasks exceeds the number of UAVs, it still falls short of DEGA's performance across all the 4 cases. CACS identifies shorter path solutions in L_20_60_1.51 and M_30_30_1.94, while DQ-EMO finds the solution with minimum completion time in S_5_40_3.95, demonstrating their respective advantages in specific scenarios. However, DEGA significantly outperforms other methods in quality of non-dominated solutions across all four instances. DEGA's superior performance across most instances can be attributed to its dual-encoding scheme, which enables simultaneous optimization of task allocation and path planning.

To further evaluate the path planning capabilities of different algorithms, we present their results for the S_5_40_3.95 instance in Figure 5. The visualization reveals that ILS, CACS, MSGA, and DQ-EMO generate relatively chaotic path planning solutions, while ACACO and DEGA find better solutions, with ACACO's approach tending to group multiple UAVs to perform certain tasks. This is reflected in Fig. 5c, where the UAVs' routes often overlap. While this approach can sometimes lead to a longer total travel distance. In short,

TABLE II: Hypervolume of All Algorithms on Test Instances

Test Cases	DEGA	noLS	noCrossover	noMutation
*L*_20_60_1.51	2.19E+09	**2.20E+09**(≈)	8.24E+08(+)	2.20E+09(≈)
*L*_80_15_0.76	**6.40E+09**	6.34E+09(+)	2.22E+09(+)	6.34E+09(+)
*L*_80_20_0.7	**1.06E+11**	1.06E+11(≈)	4.53E+10(+)	1.05E+11(+)
*L*_80_20_1.12	**3.52E+14**	3.50E+14(+)	1.37E+14(+)	3.50E+14(+)
*M*_15_30_2.16	**1.74E+15**	1.73E+15(+)	8.16E+14(+)	1.71E+15(+)
*M*_20_20_0.58	**3.14E+08**	3.13E+08(≈)	1.43E+08(+)	3.12E+08(+)
*M*_20_20_0.967	**5.21E+10**	5.21E+10(≈)	2.59E+10(+)	5.21E+10(+)
*M*_20_20_0.97	**2.65E+08**	2.61E+08(+)	9.58E+07(+)	2.62E+08(+)
*M*_30_30_1.04	**9.09E+09**	8.95E+09(+)	2.94E+09(+)	8.92E+09(+)
*M*_30_30_1.94	**2.73E+20**	2.71E+20(+)	1.18E+20(+)	2.69E+20(+)
*M*_40_10_0.67	**4.17E+09**	4.08E+09(+)	1.84E+09(+)	4.06E+09(+)
*M*_40_15_0.67	**2.43E+09**	2.40E+09(+)	1.17E+09(+)	2.40E+09(+)
*S*_10_10_3.79	**2.68E+17**	2.39E+17(+)	2.18E+17(+)	2.33E+17(+)
*S*_10_15_1.3	**1.81E+10**	1.81E+10(≈)	1.27E+10(+)	1.79E+10(+)
*S*_10_5_1.39	**5.89E+10**	5.87E+10(≈)	5.02E+10(+)	5.69E+10(+)
*S*_15_10_1.17	**2.19E+10**	2.18E+10(+)	1.24E+10(+)	2.16E+10(+)
*S*_15_20_0.67	**1.93E+07**	1.82E+07(+)	6.78E+06(+)	1.84E+07(+)
*S*_17_23_1.71	**9.65E+09**	9.45E+09(+)	6.72E+09(+)	9.35E+09(+)
*S*_20_10_0.47	**1.16E+08**	1.12E+08(+)	5.34E+07(+)	1.11E+08(+)
*S*_20_10_0.94	**3.17E+08**	3.17E+08(+)	1.78E+08(+)	3.17E+08(+)
*S*_30_10_0.65	**3.46E+08**	3.41E+08(+)	1.30E+08(+)	3.40E+08(+)
*S*_30_10_1.34	**1.51E+11**	1.49E+11(+)	6.97E+10(+)	1.48E+11(+)
*S*_3_10_1.51	4.93E+06	**5.16E+06**(−)	3.40E+06(+)	4.99E+06(≈)
*S*_3_15_5.03	**2.21E+12**	1.85E+12(+)	1.64E+12(+)	1.87E+12(+)
*S*_5_10_0.93	**7.38E+06**	7.16E+06(+)	4.70E+06(+)	7.05E+06(+)
*S*_5_10_3.67	**1.03E+13**	9.46E+12(+)	8.25E+12(+)	9.14E+12(+)
*S*_5_40_3.95	**1.72E+11**	1.65E+11(+)	9.65E+10(+)	1.61E+11(+)
*S*_5_4_0.39	**6.39E+04**	6.39E+04(≈)	3.56E+04(+)	6.39E+04(≈)
Rank Sum Test	–	20 / 7 / 1	28 / 0 / 0	25 / 3 / 0

ACACO and DEGA find better paths compared to the other four methods, as further validated by the Pareto front analysis in Fig. 4c.

D. Ablation Study

To further investigate the contribution of key components in DEGA, we conducted an ablation study by creating three modified versions of the algorithm: DEGA without crossover (noCrossover), DEGA without mutation (noMutation), and DEGA without local search (noLS). Particularly, noCrossover does not use the crossover operator; noMutation omits the mutation operator; noLS removes local search.

As shown in Table II, the full DEGA significantly outperforms its modified versions, particularly in hypervolume. The removal of any component leads to performance degradation, highlighting their critical roles. Specifically, crossover and mutation are foundational for generating diversity and exploring the solution space, while local search enhances refinement, especially when task growth rates are high relative to UAV capacity. Therefore, all three components are essential for DEGA's robust performance in multi-objective optimization.

V. Conclusion

This paper develops a dual-encoding-based genetic algorithm (DEGA) to address the multi-objective multi-UAV scheduling problem in dynamic environments. DEGA leverages a dual-encoding scheme to independently optimize the task execution order and task assignment, enhancing both flexibility and efficiency. By combining crossover, mutation, and local search, DEGA demonstrates superior performance in minimizing task completion time and travel distance compared to state-of-the-art algorithms. Experimental results on multiple instances with different scales show the effectiveness of DEGA, particularly in scenarios requiring real-time adaptation to evolving task demands.

References

[1] K. Telli, O. Kraa, Y. Himeur, A. Ouamane, M. Boumehraz, S. Atalla, and W. Mansoor, "A comprehensive review of recent research trends on unmanned aerial vehicles (uavs)," *Systems*, vol. 11, no. 8, p. 400, 2023.

[2] Y. Li, Y. Bi, J. Wang, Z. Li, H. Zhang, and P. Zhang, "Unmanned aerial vehicle assisted communication: Applications, challenges, and future outlook," *Cluster Computing*, pp. 1–16, 2024.

[3] X. Li, J. Tupayachi, A. Sharmin, and M. Martinez Ferguson, "Drone-aided delivery methods, challenges, and the future: A methodological review," *Drones*, vol. 7, no. 3, p. 191, 2023.

[4] B. Y. Lattimer, X. Huang, M. A. Delichatsios, Y. A. Levendis, K. Kochersberger, S. Manzello, P. Frank, T. Jones, J. Salvador, and C. Delgado, "Use of unmanned aerial systems in outdoor firefighting," *Fire Technology*, vol. 59, no. 6, pp. 2961–2988, 2023.

[5] S. Moon, D. H. Shim, and E. Oh, "Cooperative task assignment and path planning for multiple uavs," in *Handbook of unmanned aerial vehicles*. Springer Netherlands, 2015, pp. 1547–1576.

[6] E. Seraj, A. Silva, and M. Gombolay, "Multi-uav planning for cooperative wildfire coverage and tracking with quality-of-service guarantees," *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 2, p. 39, 2022.

[7] Y. Liu, X. Li, J. Wang, F. Wei, and J. Yang, "Reinforcement-learning-based multi-uav cooperative search for moving targets in 3d scenarios," *Drones*, vol. 8, no. 8, p. 378, 2024.

[8] A. Cardon, T. Galinho, and J.-P. Vacher, "Genetic algorithms using multi-objectives in a multi-agent system," *Robotics and Autonomous Systems*, vol. 33, no. 2-3, pp. 179–190, 2000.

[9] X.-F. Liu, Y. Fang, Z.-H. Zhan, and J. Zhang, "Strength learning particle swarm optimization for multi-objective multi-robot task scheduling," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 7, pp. 4052–4063, 2023.

[10] T. Qian, X.-F. Liu, and Y. Fang, "A cooperative ant colony system for multi-objective multi-robot task allocation with precedence constraints," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2024.

[11] G. Gao, B. Xin, Y. Mei, S. Ding, and J. Li, "A hybrid decomposition-based multi-objective evolutionary algorithm for the multi-point dynamic aggregation problem," *arXiv preprint arXiv:2105.04934*, 2021.

[12] Y. Shen and H. Li, "A multi-strategy genetic algorithm for solving multi-point dynamic aggregation problems with priority relationships of tasks," *Electronic Research Archive*, vol. 32, no. 1, pp. 445–472, 2024.

[13] X. Luo, J. Chen, Y. Yuan, and Z. Wang, "Pseudo gradient-adjusted particle swarm optimization for accurate adaptive latent factor analysis," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 4, pp. 2213–2226, 2024.

[14] S. Dai, Y.-H. Jia, W.-N. Chen, and Y. Mei, "Adaptive particle swarm optimization with local search for multi-robot multi-point dynamic aggregation," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, 2023, pp. 195–198.

[15] G. Gao, Y. Mei, Y.-H. Jia, W. N. Browne, and B. Xin, "Adaptive coordination ant colony optimization for multi-point dynamic aggregation," *IEEE Transactions on Cybernetics*, vol. 52, no. 8, pp. 7362–7376, 2021.

[16] G. Gao, Y. Mei, B. Xin, Y.-H. Jia, and W. N. Browne, "Automated coordination strategy design using genetic programming for dynamic multi-point aggregation," *IEEE Transactions on Cybernetics*, vol. 52, no. 12, pp. 13 521–13 535, 2021.

[17] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Genetic programming with lexicase selection for large-scale dynamic flexible job shop scheduling," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2023.

[18] X. Wang, T.-M. Choi, Z. Li, and S. Shao, "An effective local search algorithm for the multi-depot cumulative capacitated vehicle routing problem," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 12, pp. 4948–4958, 2019.

[19] M. Li, Z. Wang, K. Li, X. Liao, K. Hone, and X. Liu, "Task Allocation on Layered Multiagent Systems: When Evolutionary Many-Objective Optimization Meets Deep Q-Learning," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 842–855, Oct. 2021.